MERN STACK WEB DEVELOPMENT

Unit-wise Beginner Interview & Viva Questions with Answers

Units 1–5 | HTML5, CSS3, JavaScript, Node.js, Express.js, React, MongoDB and Deployment

Beginner-friendly explanations • Practical examples • MERN application flow

UNIT 1: HTML, HTML5 AND CSS3

HTML Basics

Q. What is HTML?
HTML (HyperText Markup Language) is the standard markup language used to structure content on web pages.

Q. What is the purpose of HTML in a web application?
It defines the page structure and content, such as headings, paragraphs, images, links, forms and tables.

Q. What is the basic structure of an HTML document?
A basic page contains **<!DOCTYPE html>**, **<html>**, **<head>** and **<body>**. The head stores metadata; the body contains visible content.

Q. Difference between an HTML tag and an HTML element?
A tag is the markup inside angle brackets, such as ** **. An element includes the opening tag, content and closing tag: ** Hello **.

Q. Difference between <head> and <body>?
The head contains metadata, page title, stylesheets and scripts. The body contains the content displayed in the browser.

Q. What are HTML attributes? Give examples.
Attributes provide additional information or settings for an element. Examples: **href** in a link, **src** in an image, **id**, **class** and **alt**.

Q. Purpose of the <title> tag?
It sets the page title shown in the browser tab and is also used by search engines and bookmarks.

Q. Difference between and ?
div is a block-level container, normally starting on a new line. **span** is an inline container used to style or group small portions of text.

Q. What are headings in HTML?
Headings are **<h1>** to **<h6>**. h1 is the highest-level heading and h6 is the lowest.

Q. How do you create a hyperlink?
Use an anchor element, for example: **Visit**. The href attribute gives the destination.

Forms and Tables

Q. How do you create a form in HTML?
Use **<form>** with input controls and a submit button. Example: **<form action="/login" method="post">...</form>**.

Q. Purpose of the <input> element?
It accepts user data. Its behaviour depends on the type attribute, such as text, email, password or number.

Q. Commonly used input types?
text, password, email, number, date, radio, checkbox, file, submit and tel.

Q. Difference between radio buttons and checkboxes?
Radio buttons in the same group allow one selection. Checkboxes allow independent selection of zero, one or multiple options.

Q. Purpose of the <select> element?
It creates a drop-down list. Each available choice is usually represented by an **<option>** element.

Q. Difference between GET and POST in a form?
GET sends form data in the URL query string and is commonly used to retrieve data. POST sends data in the request body and is commonly used to submit or create data. POST is not automatically secure; use HTTPS.

Q. How do you create a table?
Use **<table>** containing rows made with **<tr>**, header cells **<th>** and data cells **<td>**.

Q. Purpose of tr, th and td?
tr creates a table row; **th** creates a header cell; **td** creates a normal data cell.

HTML5

Q. What is HTML5?
HTML5 is the modern HTML standard that introduced semantic elements, native audio/video, improved forms and other web-platform capabilities.

Q. What are semantic elements?
Elements whose names describe their meaning or role, such as header, nav, main, article, section and footer.

Q. Difference between section and div?
Section represents a meaningful thematic grouping, generally with a heading. Div is a generic container with no semantic meaning.

Q. Purpose of header, nav, main and footer?
header contains introductory content; nav contains navigation links; main contains the page's primary content; footer contains closing or supplementary information.

Q. Purpose of audio and video?
They embed sound and video directly in a web page without requiring a separate media plug-in. Controls can be provided using the controls attribute.

CSS3

Q. What is CSS and why is it used?
CSS (Cascading Style Sheets) controls the appearance and layout of HTML, including colours, fonts, spacing, positioning and responsive design.

Q. Three ways of applying CSS?
Inline CSS uses the style attribute; internal CSS uses a style element; external CSS uses a separate .css file linked with link.

Q. What is a CSS selector?
A selector targets the HTML elements to which CSS rules apply. Examples include p, .card, #header and input[type="text"].

Q. What is the CSS box model?
Each element is represented by content, padding, border and margin. These layers determine the element's size and spacing.

Q. Difference between margin and padding?
Padding is space inside the border, between content and border. Margin is space outside the border, separating the element from others.

Q. What is Flexbox?
Flexbox is a one-dimensional CSS layout system used to arrange items in a row or column and control alignment, spacing and flexible sizing.

Q. What are media queries?
Media queries apply CSS based on conditions such as viewport width. They help pages adapt to phones, tablets and desktops.

UNIT 2: JAVASCRIPT, DOM AND EVENTS

JavaScript Basics

Q. What is JavaScript?
JavaScript is a programming language commonly used to make web pages interactive and to build server-side applications.

Q. Why is JavaScript used in web applications?
It handles user interactions, validation, dynamic content, API requests and application logic in browsers and on servers.

Q. Difference between JavaScript and HTML?
HTML structures content; JavaScript adds behaviour and logic to that content.

Q. Difference between JavaScript and CSS?
JavaScript implements behaviour and logic; CSS controls visual presentation and layout.

Q. What is a variable?
A variable is a named binding used to store or refer to a value.

Q. Difference between var, let and const?
var is function-scoped and can be redeclared. let is block-scoped and can be reassigned. const is block-scoped and cannot be reassigned; objects declared with const can still have their properties changed.

Q. Common JavaScript data types?
String, Number, BigInt, Boolean, Undefined, Null, Symbol and Object. Arrays and functions are also objects in JavaScript.

Q. Difference between == and ===?
== compares values after possible type conversion. === compares without type conversion, so both value and type must match.

Q. What are arithmetic, relational and logical operators?
Arithmetic operators perform calculations (+, -, *, /). Relational operators compare values (>, <, >=). Logical operators combine conditions (&&, ||, !).

Q. What is a conditional statement?
It executes different code depending on whether a condition is true or false, using if, else if, else or switch.

Loops and Functions

Q. Difference between if, if-else and switch?
if tests a condition; if-else selects between alternatives; switch compares one expression against multiple case values.

Q. What are loops?
Loops repeat a block of code while a condition is satisfied or for a specified number of iterations.

Q. Difference between for and while?
for is commonly used when initialization, condition and update are grouped together. while repeats as long as its condition is true.

Q. What is a function?
A function is a reusable block of code designed to perform a task.

Q. How do you define and call a function?
Example: **function greet() { console.log("Hello"); }** Call it using **greet();**.

Q. What are parameters and return values?
Parameters are inputs listed in a function definition. A return value is the result sent back using the return statement.

Q. What is an arrow function?
A shorter function syntax using =>, for example **const add = (a, b) => a + b;**.

Q. What is an anonymous function?
A function without a name, often assigned to a variable or passed as a callback.

Arrays, Objects, DOM and Events

Q. What is an array?
An array is an ordered collection of values, accessed by zero-based index. Example: **const colors = ["red", "blue"];**.

Q. How do you access an array element?
Use its index, such as **colors[0]**, which returns the first element.

Q. What is an object?
An object stores related data as key-value properties. Example: **const student = { name: "Ravi", age: 20 };**.

Q. Difference between array and object?
An array is mainly an indexed ordered list. An object stores values under named keys.

Q. What is DOM?
The Document Object Model is a tree-like representation of an HTML document that JavaScript can read and modify.

Q. Why is DOM manipulation required?
It lets a page respond to user actions by changing text, styles, attributes, elements or form content without manually reloading the page.

Q. How can you select an HTML element?
Common methods include **document.getElementById("id")**, **document.querySelector(".class")** and **document.querySelectorAll("p")**.

Q. Difference between getElementById and querySelector?
getElementById selects an element by its ID. querySelector accepts a CSS selector and returns the first matching element.

Q. How can you change element content?
Use **element.textContent = "New text"**. Use innerHTML only when inserting trusted HTML, because unsafe input can create security risks.

Q. What is an event?
An event is an action or occurrence detected by the browser, such as a click, key press, input change or form submission.

Q. What is event handling?
Event handling means running a function in response to an event, commonly with addEventListener().

Q. How do you validate username and password before form submission?
Read the input values, check required conditions such as non-empty fields and password length, show an error if invalid, and prevent submission when checks fail. Real authentication and security checks must also happen on the server.

Sample Viva Scenario

Q. What happens when the Login button is clicked in the given example?
The browser invokes the validate() function because it is specified in the button's onclick attribute.

Q. How does JavaScript access the entered username?
Use **document.getElementById("username").value**. The value property contains the text entered by the user.

UNIT 3: NODE.JS AND EXPRESS.JS

Node.js Fundamentals

Q. What is Node.js?
Node.js is an open-source JavaScript runtime that runs JavaScript outside the browser, commonly for servers and command-line tools.

Q. Why use Node.js in web development?
It allows JavaScript on the server, has a large package ecosystem and supports asynchronous, event-driven I/O.

Q. Can JavaScript run outside a browser?
Yes. A runtime such as Node.js provides the environment and APIs needed to execute JavaScript outside a browser.

Q. Difference between JavaScript and Node.js?
JavaScript is the language. Node.js is a runtime environment that executes JavaScript and provides server-side APIs.

Q. What is the Node.js runtime?
It is the execution environment containing the JavaScript engine and APIs that allow programs to interact with files, networks and the operating system.

Q. What is event-driven architecture?
The application responds to events such as incoming requests or completed I/O operations by invoking associated callbacks or handlers.

Q. What is non-blocking I/O?
An I/O operation can start without making the program wait for it to finish; the result is handled later through callbacks, promises or async/await.

Q. Why is Node.js suitable for web servers?
It handles many I/O-heavy connections efficiently using asynchronous operations. CPU-heavy work may need worker threads or separate services.

NPM and Modules

Q. What is NPM?
NPM is the package manager commonly used with Node.js to install, update and manage JavaScript packages.

Q. Purpose of package.json?
It stores project metadata, scripts, dependencies and other configuration used by npm.

Q. What is an npm package?
A reusable module or collection of code that can be installed and used in a project.

Q. Difference between npm install and npm init?
npm install installs dependencies (or a named package). npm init creates a package.json project manifest; npm init -y accepts defaults.

Q. What is a module in Node.js?
A module is a reusable unit of code that can expose functionality to other files.

Q. Purpose of require()?
In CommonJS, require() loads a module and returns its exported values.

Q. Difference between built-in and external modules?
Built-in modules are included with Node.js, such as fs and path. External modules are installed separately, commonly through npm.

Q. Name some built-in Node.js modules.
fs, path, http, os, events, url, crypto and stream.

File System, Server and Express.js

Q. Purpose of the fs module?
The file system module provides methods to read, write, update, delete and manage files and directories.

Q. Difference between readFile() and writeFile()?
readFile() reads file contents. writeFile() writes data to a file and creates or replaces it by default.

Q. How do you create a basic HTTP server?
Use the built-in http module: **const http = require("http");** then create a server with **http.createServer((req, res) => { res.end("Hello"); }).listen(3000);**.

Q. What is a port number?
A port is a logical endpoint used by an operating system to direct network traffic to an application or service. Example: a development server may use port 3000.

Q. Why use localhost?
localhost refers to the local computer, allowing developers to test a server on their own machine.

Q. What is Express.js?
Express is a lightweight web framework for Node.js that simplifies routing, middleware and HTTP request/response handling.

Q. Why use Express instead of only Node's http module?
Express provides convenient routing, middleware support and helper methods, reducing the amount of low-level HTTP code developers must write.

Q. How do you create an Express application?
Install Express, import it, create an app using **const app = express()**, define routes and start it with **app.listen(3000)**.

Q. What is routing in Express?
Routing maps an HTTP method and URL path to a handler function, for example app.get("/users", handler).

Q. Difference between GET and POST?
GET requests data and commonly sends parameters in the URL. POST submits data in the request body, often to create or process a resource.

Q. What are req and res?
req represents the incoming request, including parameters, query values and body. **res** is used to send the response.

Q. What is middleware?
Middleware is a function that runs during the request-response cycle. It can inspect or modify req/res, end the response, or call next() to continue.

Q. What is a REST API?
A REST-style API exposes resources through URLs and uses HTTP methods such as GET, POST, PUT, PATCH and DELETE.

Q. How do you create a route returning products as JSON?
Define **app.get("/products", (req, res) => res.json(products));**. The handler sends the products array as a JSON response.

Sample Practical Flow

Q. What happens when the user visits /products?
The browser sends a GET request to /products. Express matches the GET route, runs its handler, reads the products array and res.json() serializes and sends it as a JSON response. The browser or frontend can then display the data.

UNIT 4: REACT.JS

React Fundamentals

Q. What is React?
React is a JavaScript library for building user interfaces from reusable components.

Q. Why use React for frontend development?
It supports reusable components, declarative UI development and efficient updates when application data changes.

Q. Is React a programming language?
No. React is a JavaScript library.

Q. Is React a framework or library?
React is generally described as a UI library. Additional tools are used for routing, data fetching and other application needs.

Q. What is a Single Page Application (SPA)?
An SPA loads a main page and updates its content as users navigate or interact, often without requesting a completely new HTML page for each view.

Q. What is a component?
A component is a reusable, independent part of a user interface, such as a navbar, product card or login form.

Q. Why are components useful?
They improve reusability, organization, testing and maintainability.

Q. What is a functional component?
A JavaScript function that returns React elements, usually written using JSX.

Q. What is a class component?
A component defined using an ES6 class that extends React.Component and implements a render() method.

Q. Difference between functional and class components?
Functional components are functions and use Hooks for state and effects. Class components use lifecycle methods and this.state. Functional components are the standard approach in modern React.

JSX, Components and Props

Q. What is JSX?
JSX is a syntax extension that lets developers write HTML-like markup inside JavaScript. Build tools transform it into JavaScript React element calls.

Q. Why use JSX?
It makes component UI structure easier to read and keeps rendering logic close to the component code.

Q. Is JSX the same as HTML?
No. JSX resembles HTML but follows JavaScript syntax rules and uses some different attribute names and expression rules.

Q. Difference between class and className?
HTML uses class. JSX uses className to set the CSS class attribute.

Q. How do you write JavaScript expressions inside JSX?
Place an expression inside curly braces, for example **<h1>Hello, {name}</h1>**.

Q. Why must JSX elements be properly closed?
JSX follows XML-like syntax, so elements must be closed and a component must return one parent element or a Fragment.

Q. What are props?
Props are read-only inputs passed from a parent component to a child component.

Q. How do you pass data from parent to child?
Pass values as attributes, for example **<Student name="Ravi" />**. The child receives them through its props parameter.

Q. Can a child directly modify its props?
No. Props are read-only. The parent can pass updated values, or a child can request changes through callback props.

Q. What is component reusability?
It means using the same component in multiple places with different props instead of duplicating its implementation.

Events, Forms and Hooks

Q. How are events handled in React?
Use camelCase event props such as onClick and pass a function, for example **<button onClick={handleClick}>Click</button>**.

Q. Difference between onclick in HTML and onClick in React?
HTML commonly uses lowercase onclick. React uses camelCase onClick and expects a function rather than a string of JavaScript code.

Q. How do you handle a button click?
Define a handler function and pass it to the button's onClick prop.

Q. How do you handle input changes?
Use onChange and usually store the input value in component state with useState.

Q. How do you handle form submission?
Use onSubmit on the form, call event.preventDefault() when handling submission in the component, validate the values and then process or send the data.

Q. What are Hooks?
Hooks are React functions that let functional components use React features such as state, context and effects.

Q. What is useState()?
useState is a Hook that adds state to a functional component. It returns the current state value and a setter function.

Q. What is useEffect()?
useEffect runs side-effect logic after rendering, such as fetching data, subscribing to events or synchronizing with an external system. Its dependency array controls when it reruns.

Q. Why is state required?
State stores data that can change over time. Updating state tells React to render the UI again with the new data.

Q. What is routing in React?
Routing maps URL paths to components or views. Libraries such as React Router help users navigate between views in an SPA.

Sample Practical React Question

Q. How do you display three student names using one Student component?
Render the same component three times with different props: **<Student name="Ravi" />**, **<Student name="Priya" />** and **<Student name="Rahul" />**. Each instance displays the name it receives.

UNIT 5: MONGODB AND DEPLOYMENT

MongoDB Fundamentals and CRUD

Q. What is MongoDB?
MongoDB is a document-oriented NoSQL database that stores data in flexible BSON documents.

Q. Why is MongoDB called NoSQL?
It does not primarily use the traditional relational table-and-row model or require a fixed relational schema for every record.

Q. Difference between SQL and NoSQL?
Relational SQL databases organize data into tables with defined schemas and relationships. NoSQL databases use models such as documents, key-value pairs or graphs and often support flexible data structures.

Q. What is a MongoDB database?
A database is a logical container that holds collections.

Q. What is a collection?
A collection is a group of MongoDB documents, broadly similar to a table in a relational database.

Q. What is a document?
A document is an individual record made of field-value pairs, stored in BSON format.

Q. Difference between collection and document?
A collection groups related documents. A document is one individual record within that collection.

Q. What is BSON?
BSON (Binary JSON) is a binary-encoded serialization format used by MongoDB. It supports additional data types beyond standard JSON.

Q. What is _id?
_id is the unique primary key field for a document. MongoDB automatically generates an ObjectId when one is not supplied.

Q. Why is MongoDB suitable for web applications?
Its document model can represent application objects naturally, supports flexible schemas and provides indexing, querying and aggregation features.

Q. What does CRUD stand for?
Create, Read, Update and Delete—the four basic operations for managing data.

Q. How do you insert a document?
Use **db.students.insertOne({ name: "Ravi", age: 20 })**.

Q. How do you retrieve documents?
Use **db.students.find()** to retrieve matching documents from the collection.

Q. How do you update a document?
Use **db.students.updateOne({ name: "Ravi" }, { $set: { age: 21 } })**.

Q. How do you delete a document?
Use **db.students.deleteOne({ name: "Ravi" })**.

Q. Difference between find() and findOne()?
find() returns a cursor for all matching documents. findOne() returns the first matching document or null.

Q. What is a MongoDB query?
A query is a condition used to find documents, such as **db.students.find({ age: { $gt: 18 } })**.

Q. How can you filter documents?
Pass a filter object to find() or findOne(), using operators such as $gt, $lt, $in and $and.

Data Modelling, Mongoose and MERN Flow

Q. What is data modelling in MongoDB?
It is the process of deciding how application data is organized into documents and collections, including which fields to

embed or reference.

Q. What is an embedded document?
An embedded document is a nested document stored inside another document. It is useful for related data that is commonly read together.

Q. What is a reference between documents?
A reference stores another document's identifier, allowing related data to be stored separately and retrieved when needed.

Q. What is aggregation?
Aggregation processes documents through stages in a pipeline to transform, group, filter and calculate results. It is useful for reports and summaries.

Q. What is an index and why is it used?
An index is a data structure that helps MongoDB find documents more efficiently. It can speed up reads, but consumes storage and can add overhead to writes.

Q. How can Node.js communicate with MongoDB?
It can use the official MongoDB Node.js driver or an object data modelling library such as Mongoose to connect, query and update data.

Q. What is Mongoose?
Mongoose is an ODM library for Node.js and MongoDB. It provides schemas, models, validation and convenient database operations.

Q. What is a Mongoose schema?
A schema defines the structure, field types, validation rules and options for documents in a collection.

Q. What is a Mongoose model?
A model is created from a schema and provides an interface for creating, finding, updating and deleting documents.

Q. Explain the complete MERN data flow.
React collects user input and sends an HTTP request to the Express/Node.js backend, often using fetch or Axios. Express middleware and route handlers process the request. The backend validates data and uses Mongoose or the MongoDB driver to read or write MongoDB documents. MongoDB returns the result to the backend. Express sends a JSON response, and React updates its state and re-renders the UI.

Deployment and Practical Understanding

Q. What is deployment?
Deployment is the process of making an application available in an environment where users can access it, usually through the internet.

Q. What are the main parts to deploy in a MERN application?
Typically deploy the React frontend, the Node/Express backend and the MongoDB database or managed database service. Configure environment variables, API URLs, CORS and production settings.

Q. What is an environment variable?
A configuration value supplied outside source code, commonly used for database URLs, API keys and port settings. Secrets should not be committed to source control.

Q. What is the purpose of a build in React?
A production build compiles and optimizes frontend assets so they can be served efficiently.

Quick Revision: Full MERN Stack Flow

React (Frontend)
Displays the UI, collects user input and sends HTTP requests.

Node.js + Express.js (Backend)
Receives requests, applies middleware, validates data and runs route handlers.

Mongoose / MongoDB Driver
Connects backend code to MongoDB and performs database operations.

MongoDB (Database)
Stores application data as BSON documents inside collections.

JSON Response
Backend sends a response; React reads it, updates state and refreshes the UI.

Important viva tip: For every topic, prepare (1) a one- or two-sentence definition, (2) one real use case, and (3) a small code example where applicable. Practise explaining the flow in your own words rather than memorising definitions only.